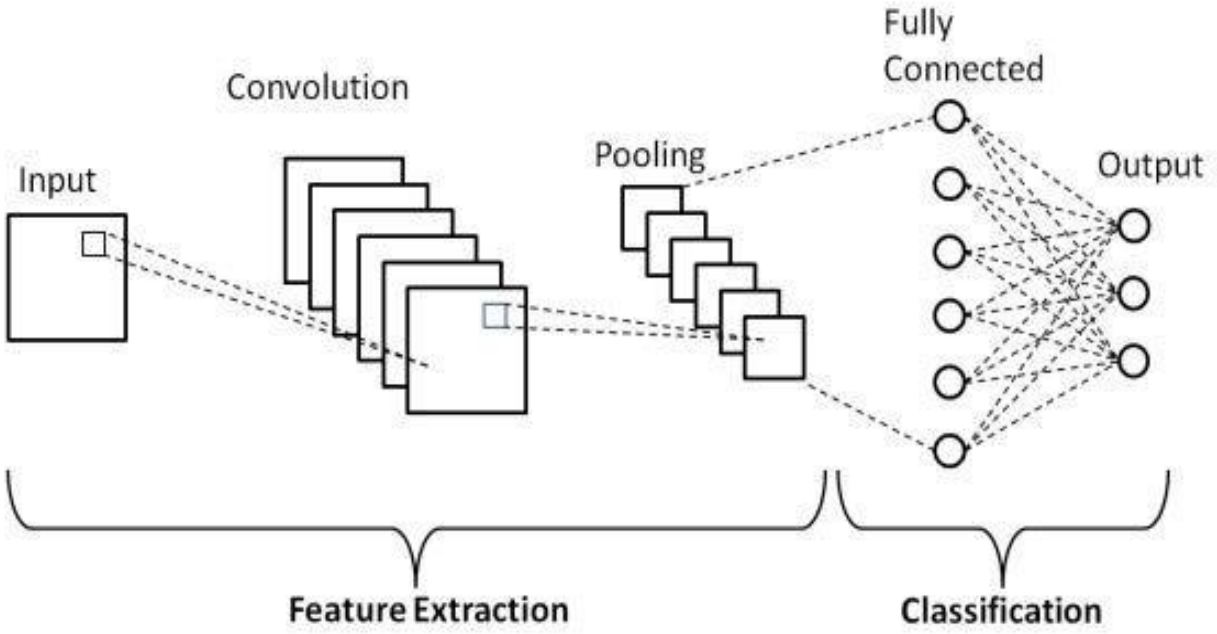
Convolutional Neural Network (CNN)

Sheenamol Yoosaf
Research Scholar
SOE,CUSAT

Convolutional Neural Network (CNN)

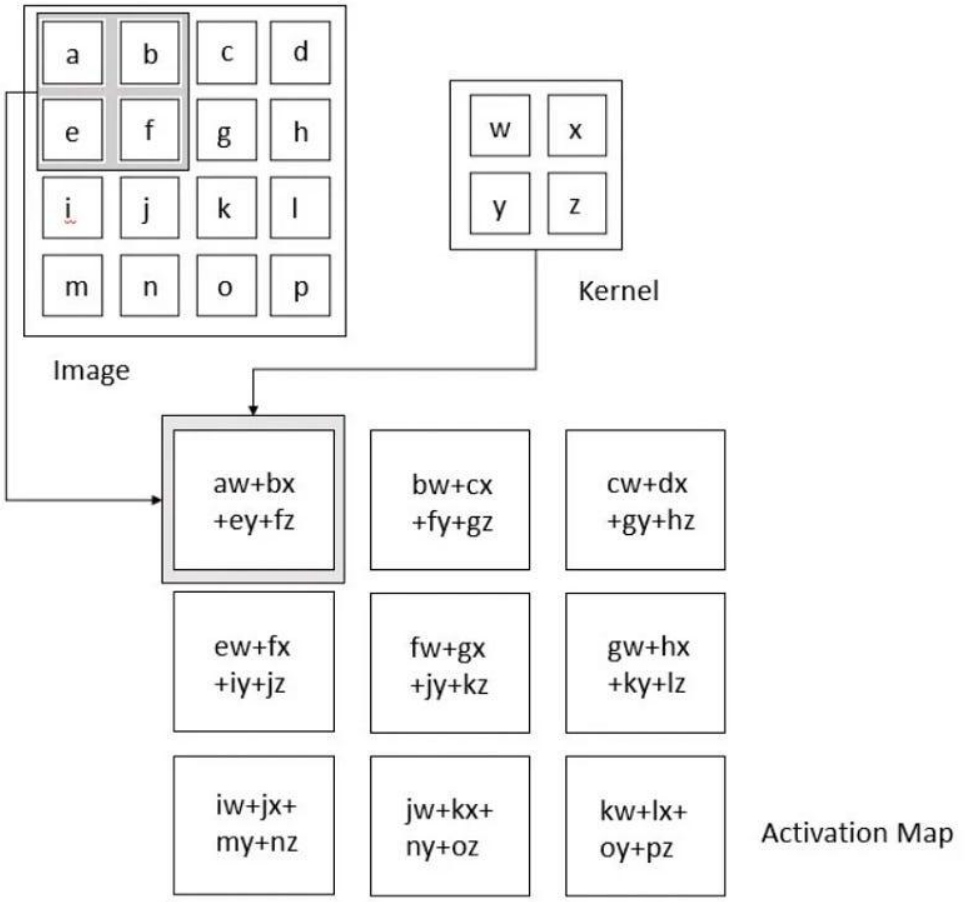


It comprises three fundamental layer types:

- Convolutional Layers
- Pooling Layers
- Fully-Connected Layers

Convolution

- Kernel size: It determines the size of the sliding window. It is generally recommended to use smaller window sizes, preferably odd values such as 1, 3, 5, and occasionally, rarely 7.
- Stride: The stride parameter determines the number of pixels the kernel window will move during each step of convolution.
- Padding: Padding refers to the technique of adding zeros to the border of an image.
- Number of filters /Depth: The number of filters in a convolutional layer determines the number of patterns or features that the layer will seek to identify.



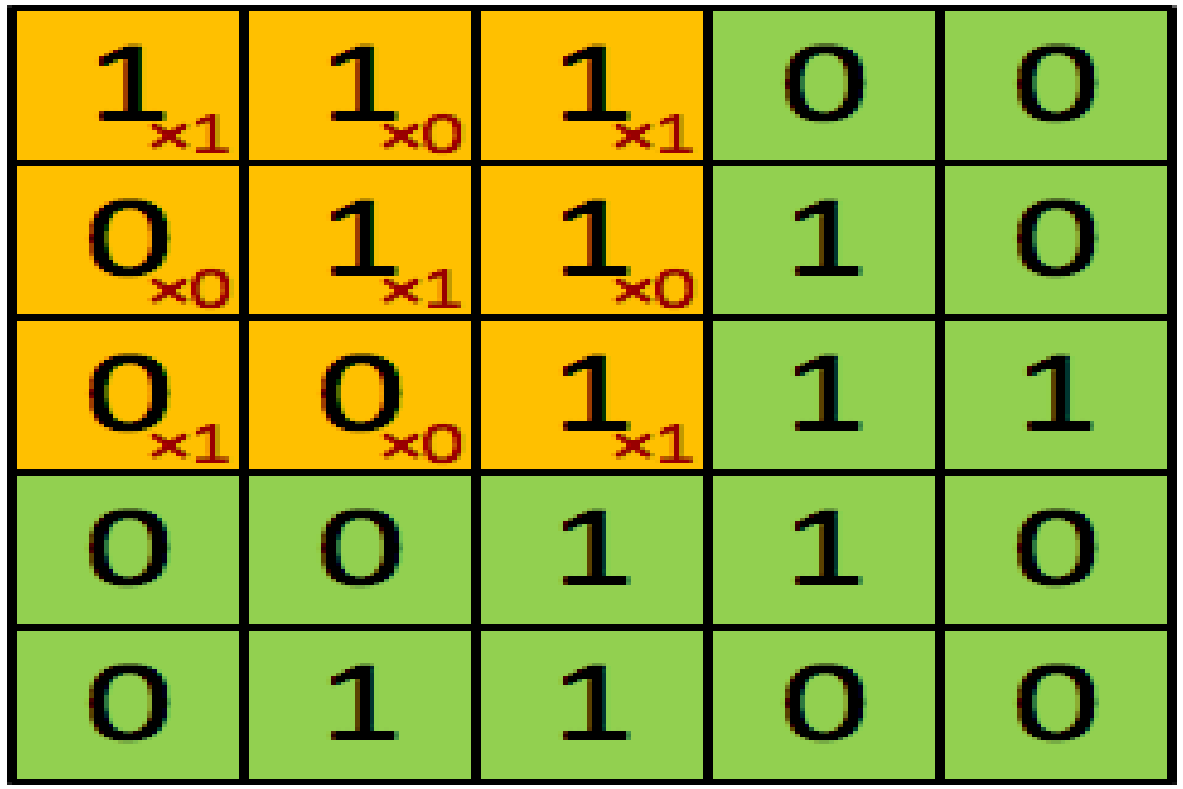
Filters

Odd-sized filters in CNNs helps to:

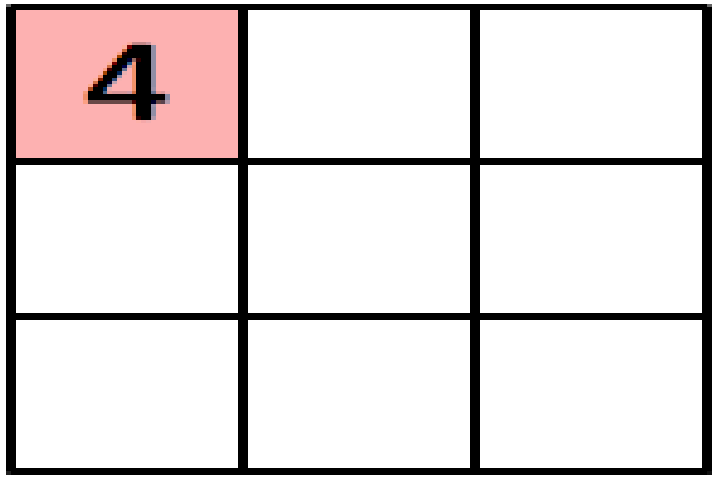
- Maintain a clear centre pixel for better focus and feature extraction.
- Simplify the padding process, ensuring that feature maps retain desired dimensions.
- Facilitate balanced and symmetric interaction with the image.
- Provide a more efficient means of detecting local and global patterns in images.

Role of Filters in the Network

- Each of the filters can be thought of as looking at the input in a different way, detecting different aspects of the input image.
- After the convolution operation, each filter produces a separate **feature map**. These feature maps are then passed on to the next layers of the network, where higher-level features are learned by combining these initial features.
- **64 filters** in a convolution layer means there are 64 **different filter matrices**, each learning different features.
- **Initial values** for the filters are set randomly, and during training, **backpropagation** adjusts the filters based on the error in predictions.
- **What features the filters learn** is determined automatically by the training process, influenced by the data and the loss function.

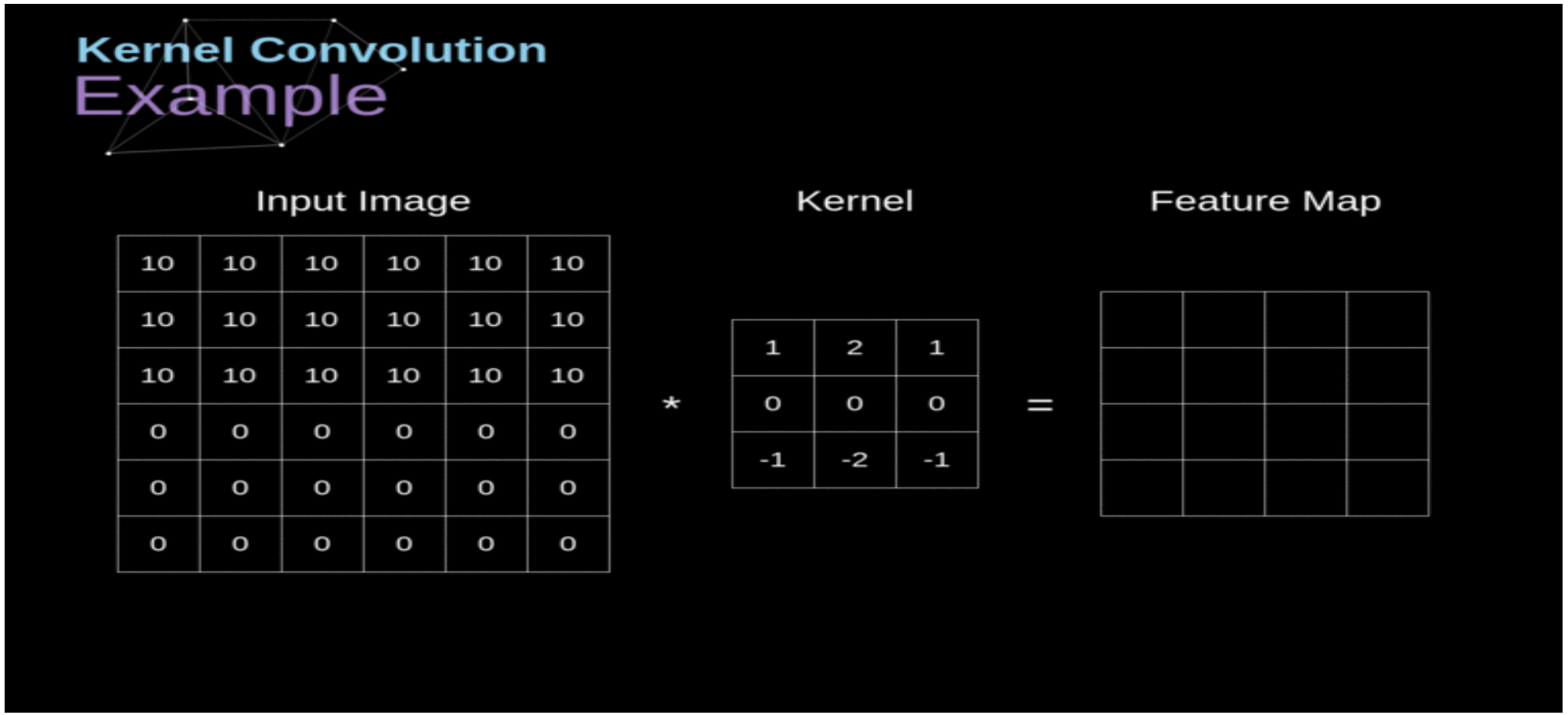


Image



Convolved Feature

Kernel Convolution Example

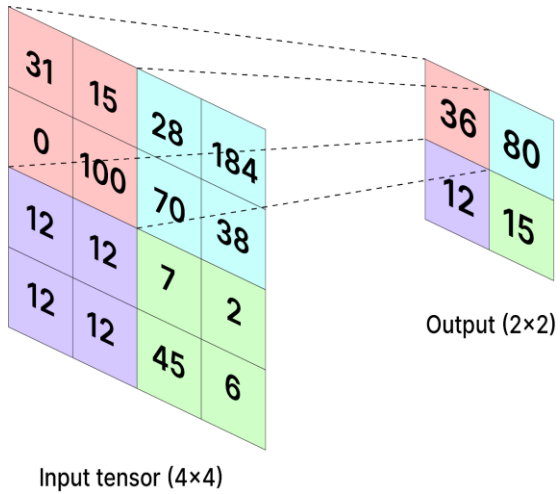


Output size of the convoluted layer

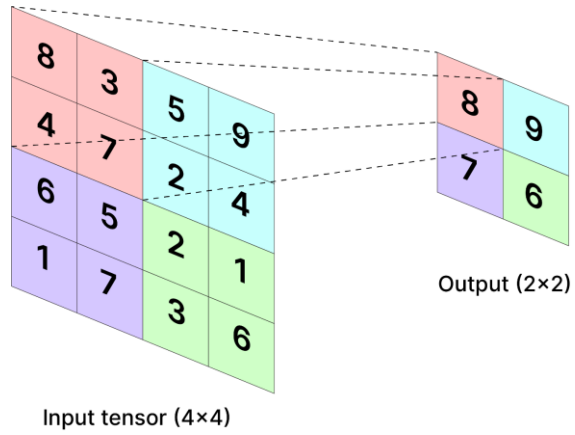
The output size of the convoluted layer is determined by several factors, including the input size, kernel size, stride, and padding

$$H_{out} = 1 + \frac{H_{in} + (2.pad) - K_{height}}{S}, \quad W_{out} = 1 + \frac{W_{in} + (2.pad) - K_{width}}{S}$$

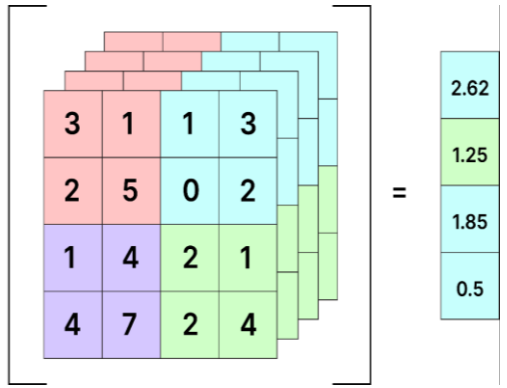
The Pooling Layer



Average pooling



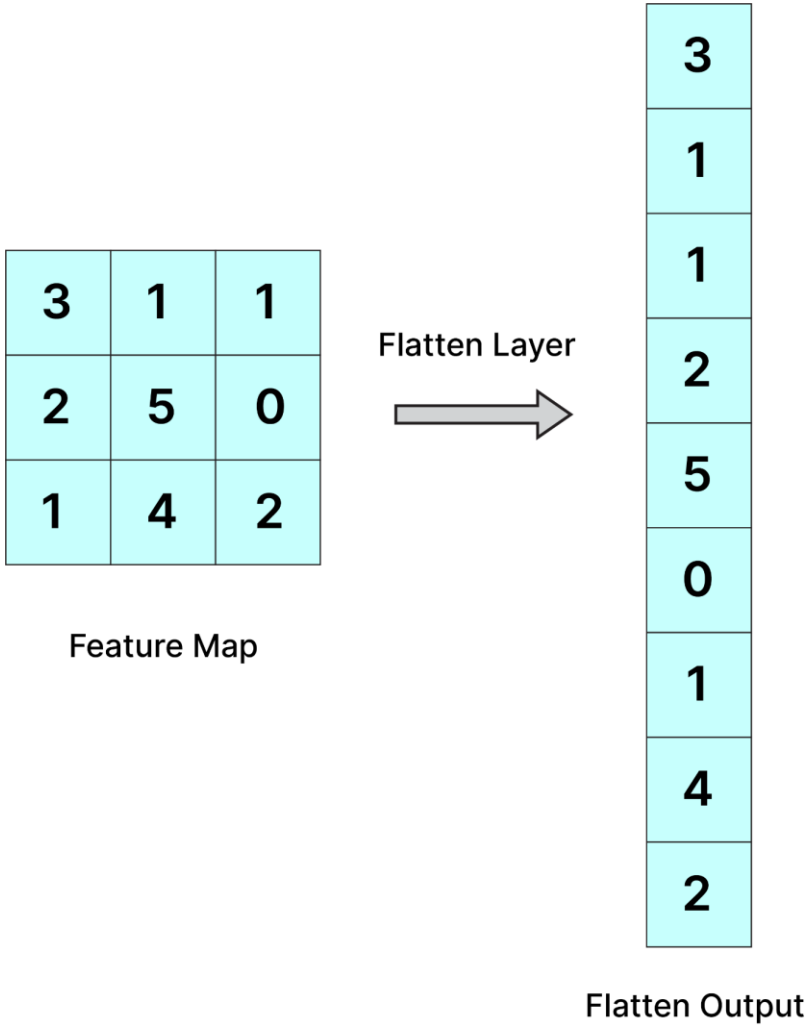
Max Pooling



global average pooling

Flatten Layer

It is used to make the multidimensional input one-dimensional, commonly used in the transition from the convolution layer to the full connected layer



Fully-Connected Layer

- The Fully Connected Layer or dense layer aims to provide global connectivity between all neurons in the layer
- The fully connected layer typically appears at the end of the ConvNet architecture, taking the flattened feature maps from the preceding convolutional and pooling layers as input
- Its purpose is to combine and transform these high-level features into the final output
- While convolutional and pooling layers tend to use ReLu functions, FC layers usually leverage a softmax activation function to classify inputs appropriately, producing a probability from 0 to 1.

Dropout

To implement this neuron de-activation,

- A dropout mask is generated (zeroes and ones) during the process of forward propagation. And this is used only during the training.
- This mask is applied to the preceding layer output or to the inputs to the next layer
- Weights are multiplied to the output i.e obtained after the mask is applied, additionally a bias is added.
- Finally, this is passed on to an activation function.

Batch Normalization

- Normalizes the inputs to a layer to have a mean of 0 and a variance of 1, improving training speed and stability.
- Calculates the mean and variance of the inputs for each batch and applies a normalization transformation.
- Commonly placed between the layers of deep networks, usually after activation functions.

$$Z = (X-M) / S$$

$$Z = Z * F$$

$$Z = Z * F + B$$

where (X), output from the activation layer, M batch mean, s standard deviation, F,B arbitrary parameter

GaussianNoise

- Adds random Gaussian noise to the inputs, acting as a form of regularization to help prevent overfitting.
- During training, a random Gaussian noise (zero-centered, with a standard deviation specified by the user) is added to the inputs.
- Often used as a noise injection layer in autoencoders or deep neural networks to improve generalization.
- GaussianNoise(0.1) adds Gaussian noise with a standard deviation of 0.1 to each input value.

Pre-trained Models

- **Pre-trained Models**: In transfer learning, models are often trained on large datasets (like ImageNet for image classification) and then repurposed. These models capture patterns from the source data that can be beneficial in new but similar contexts.

- **Feature Extraction**: In many cases, only the final layer (specific to the original task) is removed, and new layers are added for the target task. The lower layers, which capture general features, remain fixed or may be fine-tuned slightly.

- **Fine-tuning**: Some models may need fine-tuning, where a small amount of retraining is applied to adjust weights on some layers to better align with the new task while preserving general features from the original training.

Benefits

- **Data Efficiency**: Transfer learning requires less data to achieve good performance since the pre-trained model already contains learned features.

- **Faster Training**: Since much of the model is already trained, training time is reduced.

- **Improved Performance**: Transfer learning often improves performance when data is scarce or the task is complex.